Jornada 5: Bootloader y protocolo de carga de Firmware

Daniel Rosso - Pablo Madoery

Fundación Fulgor

Miércoles 23 de setiembre de 2026

Agenda

1. Motivación: ¿Por qué un bootloader?
2. Protocolo de carga UART
3. Implementación en C: boot.c
4. Ventana de boot en RTL
5. loader.py: implementación en Python
6. Robustez y manejo de errores
7. Cierre del curso: las 5 jornadas juntas
8. Lab 5
9. Lab 5 — Fase 2: integración con mailbox
10. Demo en vivo: checklist paso a paso
11. Teoría de bootloaders
12. Boot real: OTP, eFuses y raíz de confianza
13. Trabajo futuro

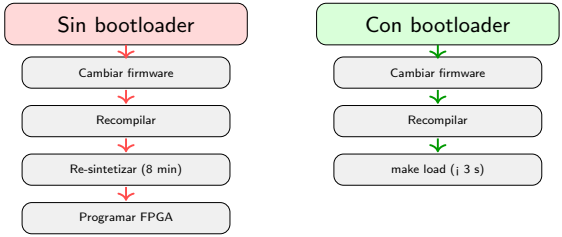
El problema: re-sintetizar para cambiar firmware

Sin bootloader:

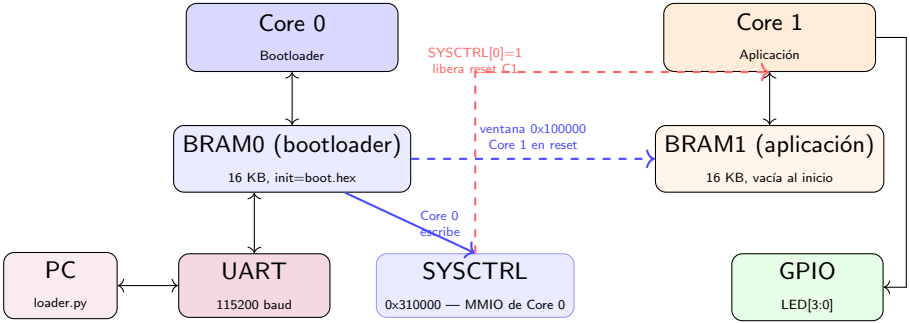
- Cada cambio en firmware de Core 1) requiere: recompilar → generar .hex → re-sintetizar → programar bitstream
- Síntesis Vivado: ∼5–10 minutos en Artix-7
- Impracticable para desarrollo iterativo

Con bootloader

- **Core 0** tiene firmware fijo embebido en síntesis (`boot.hex`)
- **BRAM de Core 1** es escribible en tiempo de ejecución
- Core 0 recibe el binario por UART y lo escribe en la BRAM de Core 1
- Ciclo completo: `make hex && make load` → **<3 seg**



Arquitectura de arranque en el SoC



Core 0 arranca desde BRAM0 (pre-cargada en síntesis). Core 1 queda en reset hasta que Core 0 le carga firmware por UART y escribe `SYSCTRL[0]=1` — `SYSCTRL` es un periférico memory-mapped visible sólo en el mapa de Core 0, no una señal directa entre los dos cores.

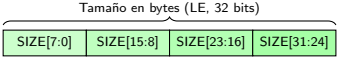
Protocolo de carga: formato de trama

Fase 0 — Sincronización:

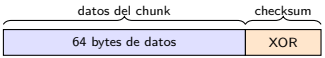


Único punto con timeout real (`rdcycle`): el host sincroniza cuando está listo, sin adivinar un `sleep()`.

Fase 1 — Tamaño (4 bytes, little-endian):



Fase 2 — Chunks de 64 bytes (repetir *N* veces):



ACK (0x06) si el checksum coincide; NACK (0x15) si no – el host reenvía el **mismo** chunk (hasta 5 veces) antes de fallar.

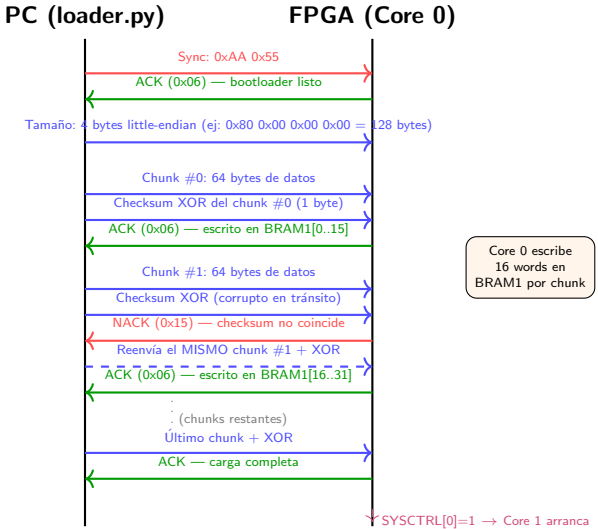
Constantes en `boot.c` y `loader.py`:

Símbolo	Valor
`SYNC_A / SYNC_B`	`0xAA / 0x55`
`ACK`	`0x06`
`NACK`	`0x15`
`CHUNK_SIZE`	64
`MAX_RETRIES`	5
`CORE1_BASE`	`0x100000`

Ventajas del diseño:

- Sincronización desacopla al host de la velocidad de arranque exacta de la FPGA
- ACK/NACK por chunk: reintenta **solo** el fallido, no toda la carga
- LE de 32 bits soporta hasta 4 GB de código de firmware

Diagrama de secuencia: carga de firmware



boot.c: función boot_load() — recepción, checksum y reintento

Constantes del protocolo:

- CHUNK_SIZE = 64 bytes (16 words)
- ACK = 0x06 / NACK = 0x15
- MAX_RETRIES = 5 por chunk

Firma:

boot_load(base, size) → 0 / -1

Código completo: lab/fw/boot.c

Qué hace, chunk a chunk (ver diagrama, slide anterior):

1. Calcula cuántos chunks de 64 bytes hacen falta para size
2. Por cada chunk: recibe los datos + 1 byte de checksum XOR calculado por el host
3. Si el XOR local coincide → **ACK**, escribe el chunk en BRAM1 (word por word) y avanza al siguiente
4. Si no coincide → **NACK**; el host reenvía el **mismo** chunk (hasta MAX_RETRIES veces)
5. Se agotan los reintentos de un chunk → retorna -1 (error); todos los chunks OK → retorna 0

boot.c: main() y reset diferido de Core 1

Flujo de `main()` en Core 0:

1. Enciende LED[0] (bootloader activo) y anuncia el banner por UART
2. Espera la sincronización (`0xAA 0x55`) con **timeout real** — único punto del protocolo con timeout
3. Lee el tamaño (4 bytes little-endian) y llama a `boot_load(CORE1_BASE, size)`
4. **Éxito**: informa bytes cargados y libera el reset de Core 1 escribiendo `SYSCTRL`
5. **Error**: reporta la falla y queda en loop infinito (Core 1 nunca arranca)

Registro SYSCTRL (`0x310000`):

- bits `[5:4]`: LED de estado del bootloader
- bit `[0]`: `REL_C1` — libera el reset de Core 1

Por qué el reset es diferido

Si Core 1 arrancara antes de tener firmware válido en su BRAM, fetchearía instrucciones arbitrarias de una memoria vacía. Core 0 solo escribe `REL_C1` después de confirmar que `boot_load()` devolvió éxito.

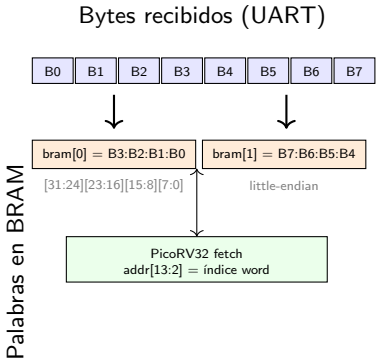
boot.c: escritura word por word en BRAM

¿Por qué word por word (4 bytes)?

- RISC-V es una ISA de 32 bits: cada instrucción ocupa 4 bytes
- La BRAM está organizada en words de 32 bits
- PicoRV32 usa `mem_addr[13:2]` como índice ⇒ byte 0 = palabra 0
- Hay que respetar el orden **little-endian**: byte 0 va en bits [7:0] de la palabra

Ventana de boot en `soc_boot.v`:

- Core 0 escribe en `0x100000+offset`
- **¿Qué es?** Rango `0x100000–0x103FFF` en el mapa de Core 0 que da acceso de solo escritura al puerto de escritura de BRAM1, mientras Core 1 está en reset — no es una memoria propia
- Cuando `c1_resetn_r == 1`: BRAM1 pasa a control exclusivo de Core 1, que la ve en su propio mapa como `0x000000`

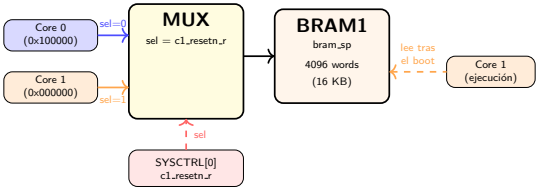


soc_boot.v: ventana de boot mediante mux

¿Qué es la ventana de boot? Rango `0x100000–0x103FFF` en el mapa de Core 0 que da acceso de solo escritura al puerto de escritura de BRAM1, mientras Core 1 está en reset. No es una memoria propia: es un mux que decide quién controla el puerto físico de BRAM1.

El problema de dos masters en una BRAM:

- BRAM1 es accedida por **dos masters** en momentos distintos:
  - Core 0 durante la carga (`c1_resetn_r=0`)
  - Core 1 durante la ejecución (`c1_resetn_r=1`)
- Solución: **mux** en las entradas de la BRAM controlado por `c1_resetn_r`
- No se necesita dual-port: los dos accesos son mutuamente exclusivos en el tiempo



soc_boot.v: implementación del mux

El mux de `bram1_*` **en** `soc_boot.v` — cuatro señales combinacionales seleccionan, para cada entrada de BRAM1, si el dato viene de Core 0 (ventana de boot) o de Core 1 (ejecución), según `c1_resetn_r`:

```
// Selector: Core 1 en reset -> Core 0 escribe
wire bram1_ena =
    c1_resetn_r ? c1_sel_bram : c0_sel_boot;
wire [3:0] bram1_wea =
    c1_resetn_r ? c1_wstrb
                : (c0_sel_boot ? c0_wstrb : 4'b0);
wire [11:0] bram1_addr =
    c1_resetn_r ? c1_addr[13:2] : c0_addr[13:2];
wire [31:0] bram1_din =
    c1_resetn_r ? c1_wdata : c0_wdata;
```

Mientras `c1_resetn_r == 0` (Core 1 en reset), las cuatro señales toman su rama izquierda: `bram1_*` queda controlado por Core 0 a través de `c0_sel_boot` (la ventana de boot). Apenas `c1_resetn_r` pasa a 1, todas conmutan juntas a la rama derecha y BRAM1 queda exclusivamente bajo control de Core 1.

loader.py: sincronización y reintento por chunk

`wait_sync_ack()`

- Envía `SYNC = 0xAA 0x55`
- Espera 1 byte de respuesta; éxito solo si es `ACK (0x06)`

`load_firmware(ser, firmware)` — espejo exacto, del lado del host, de `boot_load()` en la FPGA (slide anterior):

- Sincroniza y envía el tamaño (4 bytes LE)
- Parte el binario en chunks de `CHUNK_SIZE=64` bytes

Por cada chunk (hasta `MAX_RETRIES=5`):

1. Calcula el XOR de los 64 bytes
2. Envía chunk + XOR
3. `ACK` → siguiente chunk
4. `NACK` → reenvía el **mismo** chunk
5. Timeout o reintentos agotados → aborta (`False`)

Código completo: `host/loader.py`.

loader.py: --parse — releyendo la Jornada 1 en el binario real

```
python3 host/loader.py --bin fw/app_alt.bin --parse
```

```
00000C  00004117  .A..  U (auipc)
000010  FF410113  ..A.  I (op-imm)
00001C  00B57863  cx..  B (branch)
000020  00052023  # ..  S (store)
0000CC  00CB0AB3  ....  R (op / mul-div)
```

Makefile: cargar y recargar en caliente

```
# Cargar app.bin en el FPGA via UART -- termina solo, libera el puerto
load: fw/app.bin
    python3 host/loader.py --port $(PORT) --baud $(BAUD) --bin fw/app.bin

# Recargar en caliente con la variante -- sin resintetizar
load-alt: fw/app_alt.bin
    python3 host/loader.py --port $(PORT) --baud $(BAUD) --bin fw/app_alt.bin

# Consola serie en vivo -- aparte, nunca junto a load/load-alt
monitor:
    python3 host/loader.py --port $(PORT) --baud $(BAUD) --monitor-only
```

Flujo típico de desarrollo:

1. `make synth` (primera vez, ∼8 min)
2. `make program` (programar FPGA, 30 s)
3. `make load` – BER con tasa de error fija (patrón de LEDs estático)
4. `make load-alt` – misma FPGA, barrido de BER, sin resintetizar
5. Se pueden encadenar `load`/`load-alt`

Puerto customizable:

- `PORT=/dev/ttyUSB0 make load`
- En Linux: agregar usuario al grupo `dialout`
- `sudo usermod -aG dialout $USER`

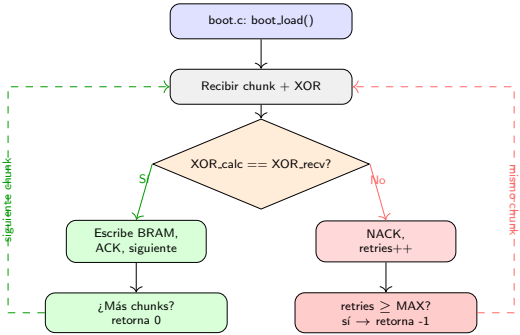
Robustez: ¿qué pasa si la carga falla?

Escenarios de falla:

1. **Host nunca arranca:** `wait_sync()` tiene timeout real (`rdcycle`): a los 5 s imprime error y frena – no hace falta desenchufar la placa.
2. **Checksum incorrecto:** NACK y el host reenvía el **mismo** chunk (hasta `MAX_RETRIES`). Ya no aborta al primer error.
3. **Desconexión a mitad de un chunk:** Core 0 queda bloqueado en `uart_getc()` (sin timeout ahí, ver próxima slide). Necesita reset manual.
4. **Chunk siguiente sin esperar ACK:** entraría mientras la UART no tiene FIFO ⇒ overflow. El protocolo ya espera ACK/NACK antes de continuar.

Mejoras posibles (no implementadas):

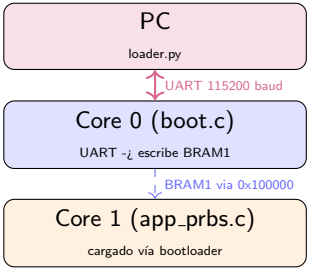
- Timeout también dentro de un chunk ya iniciado
- CRC-16 en lugar de XOR (ver comparación más adelante)



El bootloader, último eslabón del recorrido

¿Qué habilita el bootloader?

- **Desarrollo rápido:** cambiar la aplicación de Core 1 (otra tasa de error PRBS) sin re-sintetizar
- **Separación de responsabilidades:**
  - Core 0 = infraestructura (boot + UART)
  - Core 1 = aplicación PRBS/BER cargada dinámicamente
- **Recorrido completo del curso:** de decodificar una instrucción a mano (Jornada 1) a recargar en caliente el firmware que mide el BER de un enlace, sin tocar el bitstream



Después: C0 espera,
C1 mide BER y muestra el resultado en LEDs

Lab 5: implementar el bootloader desde el esqueleto

Objetivo

Completar los TODO para que `make sim` muestre `Core 1 activo --- boot exitoso!` y `make load` cargue `app.bin` (PRBS/BER) en hardware real. Verificación final: `make load-alt` cambia el patrón de LEDs sin resintetizar – esa es la recarga en caliente.

Tareas de firmware (`fw/boot.c`):

1. **TODO 1 — boot_load()**:
   - Loop de recepción: `uart_getc()` `len` veces, acumular XOR
   - Recibir checksum del host y comparar
   - Ensamblar words little-endian y escribir en `base[]`
2. **TODO 2 — recibir tamaño**: 4 bytes LE con `uart_getc()`
3. **TODO 3 — main()**: llamar `boot_load()`, imprimir resultado, liberar Core 1 vía SYSCTRL

Ya dados: `wait_sync()` (handshake AA 55 → ACK con timeout) y el reintento por NACK en `boot_load()`.

Tareas de RTL (`rtl/soc_boot.v`):

1. **TODO 1 — mux BRAM1**: señales de mux (`bram1_ena/wea/addr/din`) e instanciar `u_bram1`
2. **TODO 2 — ready Core 0**: `always` para `c0_bram_rdy_r` y `c0_boot_rdy_r`
3. **TODO 3 — ready Core 1**: `always` para `c1_bram_rdy_r`

Fase 2: bootloader + mailbox en el mismo SoC

Motivación

En el lab anterior, Core 0 queda ocioso (`while(1)`) apenas libera a Core 1. Sin embargo, el rol de *manager* de la Jornada 4 (generar PRBS, coordinar por mailbox) desaparece. El objetivo es fusionar `soc_boot.v` (ventana de boot) con `soc_2core.v` (mailbox) en un solo `soc_mboot.v`.

Tareas de RTL (`rtl/soc_mboot.v`):

1. **TODO A** — decodificación `c0_sel_mbx`/`c1_sel_mbx` (rango 0x500000)
2. **TODO B** — instanciar `bram_dp` para el mailbox
3. **TODO C/D** — `mem_ready` Core 0: registrar y sumar a `c0_ready`
4. **TODO E** — mismo patrón para Core 1
5. **TODO F/G** — sumar `mbx_rdata0/1` al mux de `mem_rdata` de cada core

Tarea de firmware (`fw0/main.c`):

1. **TODO 1** — encadenar `wait_sync()` + `load_and_release()` antes de entrar al loop de manager

`wait_sync()`/`load_and_release()` (boot, J5) y el loop de manager con recarga en caliente (J4) vienen dados completos.

Demo en vivo: checklist desde el power-on

Programar y cargar:

1. `make program` – `openFPGALoader`
2. `make load` – `app.bin`: termina solo y libera el puerto; patrón de LEDs **fijo** (BER constante)
3. `make load-alt` – recarga en caliente `app_alt.bin`: barrido triangular de BER $\Rightarrow$ los LEDs **recorren los 4 patrones en vivo** ($\sim$1 s por ciclo), **sin resintetizar**
4. Repetir `load` $\leftrightarrow$ `load-alt` las veces que haga falta, sin cortar nada a mano entre medio
5. (Opcional, aparte) `make monitor` – consola serie en vivo

¿Por qué `make hex` genera dos binarios de una sola fuente?

Un solo `.c`, dos compilaciones:

```
fw/app.elf: fw/app_prbs.c
    $(CC) $(CFLAGS) fw/app_prbs.c -o $@
    # sin -D: usa los defaults del .c

fw/app_alt.elf: fw/app_prbs.c
    $(CC) $(CFLAGS) \
      -DSWEEP_BER=1 \
      -DDELAY_CYCLES=1000000u \
      fw/app_prbs.c -o $@
    # -D pisa los defaults en compile-time
```

El truco en `app_prbs.c`:

```
#ifndef ERR_PERIOD_BITS
#define ERR_PERIOD_BITS 1021u
#endif
#ifndef DELAY_CYCLES
#define DELAY_CYCLES 5000000u
#endif
#ifdef SWEEP_BER
  /* barre el periodo de error entre
     SWEEP_LO y SWEEP_HI cada ventana */
#endif
```

- `make hex` corre las dos reglas en la misma invocación – no hay que tocar el `.c` entre una y otra
- `app.elf`: tasa de error **fija** ⇒ bucket de BER constante, patrón de LEDs estático
- `app_alt.elf`: periodo de error **en barrido** ⇒ BER sube y baja solo, LEDs recorren los 4 buckets en vivo
- Por eso la recarga en caliente no resintetiza: son 2 `.bin` distintos, cargados por UART a demanda

Resumen de la Jornada 5

Conceptos vistos:

- Motivación del bootloader: separar firmware de síntesis
- Sincronización AA 55 → ACK con timeout real (`rdcycle`)
- Protocolo: tamaño LE + chunks con ACK/NACK por chunk
- NACK + reintento del mismo chunk (`MAX_RETRIES`)
- Checksum XOR por chunk (no global): acota el error a 1 bloque
- Escritura word-por-word en BRAM: ensamble little-endian
- Ventana de boot: mux en BRAM1 vía `c1_resetn_r`
- Reset diferido; `loader.py` con `pyserial`; UART sin FIFO

Archivos del proyecto:

- `fw/boot.c` — bootloader Core 0
- `fw/app_prbs.c` — aplicación Core 1 (PRBS/BER en LEDs)
- `fw/prbs.h` — LFSR generador/checker (Jornadas 4 y 5)
- `fw/uart.h` — driver UART bare-metal
- `rtl/soc_boot.v` — SoC con ventana de boot
- `rtl/soc_boot_top.v` — wrapper Arty A7
- `host/loader.py` — cliente Python
- `sim/tb_soc_boot.v` — testbench iverilog
- `Makefile` — targets: `hex`, `sim`, `load`, `load-alt`

Backup Slides

Material adicional de soporte

Bootloaders en sistemas embebidos I

Etapas de arranque (stages):

- **Stage 1 — ROM bootloader**: inmutable, quemado en el chip. Solo inicializa DRAM/SRAM y llama al Stage 2. En microcontroladores (STM32, ESP32): los últimos 16–64 KB de flash. En ARM SoC: ROM interna del fabricante.
- **Stage 2 — Software bootloader**: cargado desde flash/UART/SD. Puede verificar firma, seleccionar imagen, actualizar firmware. Ejemplos: U-Boot (Linux embebido), Das U-Boot, GRUB, nuestro `boot.c`.
- **Stage 3 — aplicación/SO**: el firmware de usuario que bootloader carga en RAM y ejecuta.

Cadena de arranque en distintos sistemas:

Sistema	Stage 1	Stage 2	Stage 3
PC x86	BIOS/UEFI	GRUB	Linux kernel
Raspberry Pi	ROM BCM2711	bootcode.bin	Linux
STM32	ROM (UART/USB)	aplicación	—
ESP32	ROM bootloader	ESP-IDF bootloader	app
Nuestro SoC	BRAM0 (síntesis)	boot.c	app_prbs.c / app de C1

Bootloaders en sistemas embebidos II

Requisitos de un bootloader robusto:

1. **Atomicidad**: si se interrumpe la carga, el sistema no queda en estado inválido. Estrategia: escribir nuevo firmware en área separada, solo conmutar el puntero de ejecución cuando la verificación pasa. En nuestro SoC: si falla el checksum de un chunk, se envía NACK y Core 1 nunca sale de reset — la BRAM de destino queda parcialmente escrita pero inutilizada.
2. **Verificación de integridad**: checksum, CRC-32, o hash criptográfico. Cuanto más fuerte, más overhead de cómputo y tiempo.
3. **Mecanismo de fallback**: si la imagen es inválida $\rightarrow$ modo de espera, imagen de fábrica, o reset loop.
4. **Timeout**: si no llega firmware en N segundos, continuar con imagen anterior o modo safe.

OTP / eFuses: la raíz de confianza antes del Stage 1

El Stage 1 (ROM bootloader) es inmutable de fábrica, pero necesita datos que sí cambian por chip individual:

- Clave pública (o su hash) para verificar la firma del Stage 2
- Modo de arranque por defecto
- ID de chip / lote, calibración analógica
- Contador anti-rollback (versión mínima de firmware aceptada)

Esos datos viven en **OTP (One-Time Programmable) / eFuses**: bits físicos que se **queman una sola vez** (un fusible microscópico se abre y no vuelve a cerrar) – no es flash regular.

¿Por qué no alcanza con flash normal?

Flash se puede re-flashear – incluso por alguien con acceso físico y sin credenciales. Un fusible quemado es **irreversible**: es la única raíz de confianza que ni el propio firmware puede alterar en tiempo de ejecución.s

Nuestro `BRAM0` (Jornada 5) **no** es esto: se recarga en cada síntesis, cualquiera con Vivado la reescribe.

Selección de fuente de arranque: pines de strap + OTP

El ROM lee, al reset, una combinación de:

- **Pines de strap** (hardware, fijos por diseño de PCB o jumpers): `BOOT0`/`BOOT1` en STM32, `GPIO0`/`GPIO2`/`GPIO5` en ESP32
- **Bits en OTP**: modo de arranque grabado en fábrica o por el fabricante del equipo final

Con eso decide: ¿cargar desde flash SPI interna? ¿esperar UART/USB (modo DFU/download)? ¿leer de SD/eMMC?

Sistema	Stage 1	Raíz de confianza	Selección
PC x86	BIOS/UEFI	TPM (a veces externo)	orden en NVRAM
Raspberry Pi	ROM BCM2711	OTP (secure boot, Pi 4+)	`config.txt`
STM32	ROM interna	Option bytes (OTP)	pines BOOT0/1
ESP32	ROM bootloader	eFuse (hash clave, flash-enc)	GPIO strapping
Nuestro SoC	BRAM0 (síntesis)	**ninguna**	fijo: siempre C0

Nuestro SoC no tiene selección de fuente: Core 0 siempre arranca de BRAM0 y siempre espera UART – no hay modo alternativo ni OTP que lo condicione.

Secure boot en la práctica: ESP32 y la propia Arty A7

Cadena de verificación en ESP32 (Secure Boot V2):

1. El ROM calcula SHA-256 de la clave pública grabada en eFuse
2. La compara contra el hash quemado (no la clave completa: ahorra espacio en OTP)
3. Verifica la firma RSA del Stage 2 (bootloader) con esa clave
4. Firma inválida → no arranca (o entra en modo recovery), nunca ejecuta el Stage 2

Esto no es ajeno a nuestra placa:
El propio **Artix-7** de la Arty A7 tiene **eFuses OTP reales** de usuario:

- Clave AES-256 para **cifrar el bitstream** (protege el diseño contra copia)
- Bits para deshabilitar JTAG/readback tras programar

Irreversible de verdad

Vivado los graba con la opción `-eFUSE` del bitstream. Si se programa mal la clave, **ese chip queda inutilizable** para ese propósito – no hay forma de deshacerlo.

Nuestro diseño educativo no usa nada de esto: `soc_boot.v` no verifica firma alguna antes de liberar Core 1 – la raíz de confianza real es simplemente "quién tiene el archivo `.bit`".

Trabajo futuro: arquitectura del SoC

Cómputo:

- **Co-procesador PCPI** para PicoRV32: acelerar en hardware el checksum o el LFSR de PRBS
- **Otro core (CVA6)**: más capacidad de cómputo, target posible para Linux/FreeRTOS
- **Caché de instrucciones** para ejecutar desde DDR/QSPI sin morir en latencia
- **Mutex / semáforos en hardware** para sincronizar los cores sin depender del protocolo del mailbox

Sistema:

- **Interconnect estándar** en lugar del mux ad-hoc de `soc_boot.v` (ver slides siguientes)
- **DMA UART → BRAM1**: saca a Core 0 del copy loop byte a byte del bootloader
- **Timer + interrupciones** (estilo CLINT/PLIC): PicoRV32 soporta IRQ; reemplaza el polling de la UART y del mailbox
- **DDR3 de la Arty A7** (256 MB) vía MIG: su interfaz es AXI4, otra razón concreta para tener un interconnect
- **Watchdog**: si Core 1 se cuelga, Core 0 lo resetea y recarga la imagen

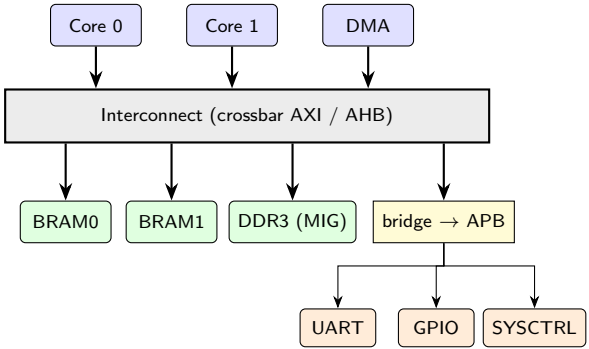
Trabajo futuro: agregar un interconnect

Hoy: decodificación ad-hoc en `soc_boot.v`

- Un `c0_sel_*`/`c1_sel_*` por región, comparando bits de `addr` a mano
- `ready` y `rdata` armados con OR/mux escritos a mano
- El "arbitraje" de BRAM1 es `c1_resetn_r`: C0 o C1, nunca los dos
- Cada periférico nuevo = tocar decodificador, mux de lectura y lógica de `ready`

Con un interconnect:

- Mapa de memoria **parametrizado** (tabla base/máscara)
- **Arbitraje** real: varios masters (C0, C1, DMA) acceden al mismo slave
- Periféricos reutilizables: cualquier IP con la interfaz estándar se enchufa sin reescribir nada



Punto de partida: `picorv32.v` ya trae los wrappers `picorv32_axi` (AXI4-Lite) y `picorv32_wb` (Wishbone).

Trabajo futuro: ¿qué bus? AMBA APB / AHB / AXI y alternativas

Bus	Uso típico	Cómo transfiere	Bursts / pipeline
APB	periféricos lentos (UART, GPIO, timers)	2 fases setup/access	no / no
AHB(-Lite)	bus de sistema de MCUs (Cortex-M)	dirección y dato solapados	sí / 1 en vuelo
AXI4	DDR, SoCs de alto rendimiento, Zynq	5 canales valid/ready	hasta 256 beats / muchas, con IDs
AXI4-Lite	registros de control de IP en FPGA	5 canales, 1 beat	no / limitado
AXI4-Stream	flujos sin dirección (PRBS, DSP, video)	`tvalid/tready/tdata`	flujo continuo
Wishbone	cores abiertos (OpenCores, LiteX)	`cyc/stb/ack`	bloques / modo pipelined
OBI	cores OpenHW (CV32E40P, CVE2)	`req/gnt` + `rvalid`	no / sí
TileLink	SoCs Chisel (Rocket, BOOM)	canales A/D (+B/C/E coherencia)	sí / sí

Combinación habitual (y la que tendría sentido acá)

AXI4-Lite (o AXI4 si entra la DDR3) como bus de sistema entre cores y memorias → **bridge a APB** para los periféricos de registros. El generador de PRBS y el checker de BER serían candidatos naturales a **AXI4-Stream**.

Trabajo futuro: boot, protocolo y seguridad

Conectividad y fuentes de boot:

- **Ethernet**: carga de firmware por red y telemetría de BER en vivo sin cable serie
- **Boot desde SD** como Stage 2 alternativo al UART actual
- **Persistencia en flash QSPI** de la Arty A7: hoy todo lo cargado por UART se pierde al cortar alimentación

Seguridad:

- **Bitstream encriptado** (AES con clave en eFuse/BBRAM): resuelve de raíz "la raíz de confianza es quién tiene el `.bit`"

Protocolo y robustez:

- **CRC-32** real en lugar del checksum XOR del protocolo de carga
- **Imagen de fallback** pre-cargada en síntesis con timeout, para no depender de que alguien conecte un host
- **Inyección de errores** real desde `loader.py` (ruido controlado en la UART) para medir NACK/retry contra errores de verdad, no solo simulados con LFSR

Trabajo futuro: verificación y herramientas

Verificación:

- **cocotb + Verilator**: testbenches corriendo en CI en cada push
- **Verificación formal** (SymbiYosys) del interconnect: propiedades del handshake (`valid` no baja sin `ready`, datos estables, etc.)
- **ILA** de Vivado sobre el bus para ver transacciones en hardware

Herramientas:

- **Debug JTAG en vivo** de los PicoRV32 (`openocd`), una vez resuelto el tema de la sonda externa en esta placa
- **Contadores de performance** (`rdcycle`, `rdinstret`) para medir CPI y el costo real del bootloader
- **Flujo 100 % abierto**: Yosys + nextpnr-xilinx / F4PGA
- **LiteX**: generar el SoC y compararlo contra el diseño hecho a mano